

NMAP



NMAP COMMANDS

Basic Scanning Techniques

- Scan a single target
- Scan multiple targets
- Scan a list of targets
- Scan a range of hosts
- Scan an entire subnet
- Scan random hosts
- Excluding targets from a scan
- Excluding targets using a list
- Perform an aggressive scan
- Scan an IPv6 target

```
nmap [target]
nmap [target1,target2,etc]
nmap -iL [list.txt]
nmap [range of IP addresses]
nmap [IP address/cdir]
nmap -iR [number]
nmap [targets] -exclude [targets]
nmap [targets] -excludefile [list.txt]
nmap -A [target]
nmap -6 [target]
```

```
# nmap -sp 192.100.1.1/24
```

This ping scan command returns a list of hosts on network and the total number of assigned IP

```
# nmap -O <target IP>
```

When using this type of scan, Nmap sends TCP and UDP packets to a particular port, and then analyze its response. It compares this response to a database of 2600 operating systems, and return information on the OS (and version)

NMAP SCANNING EXAMPLES

Scan IP address (Targets)

Command	Description
<code>nmap 10.0.0.1</code>	Scan a single host IP
<code>nmap 192.168.10.0/24</code>	Scan a Class C subnet range
<code>nmap 10.1.1.5-100</code>	Scan the range of IPs between 10.1.1.5 up to 10.1.1.100
<code>nmap -iL hosts.txt</code>	Scan the IP addresses listed in text file "hosts.txt"
<code>nmap 10.1.1.3 10.1.1.6 10.1.1.8</code>	Scan the 3 specified IPs only
<code>nmap www.somedomain.com</code>	First resolve the IP of the domain and then scan its IP address

NMAP SCANNING EXAMPLES

Port Related Commands

Command	Description
<code>nmap -p80 10.1.1.1</code>	Scan only port 80 for specified host
<code>nmap -p20-23 10.1.1.1</code>	Scan ports 20 up to 23 for specified host
<code>nmap -p80,88,8000 10.1.1.1</code>	Scan ports 80,88,8000 only
<code>nmap -p- 10.1.1.1</code>	Scan ALL ports for specified host
<code>nmap -sS -sU -p U:53,T:22 10.1.1.1</code>	Scan ports UDP 53 and TCP 22
<code>nmap -p http,ssh 10.1.1.1</code>	Scan http and ssh ports for specified host

NMAP COMMANDS

Discovery Options

- | | |
|----------------------------------|---|
| • Perform a ping scan only | <code>nmap -sP [target]</code> |
| • Don't ping | <code>nmap -PN [target]</code> |
| • TCP SYN Ping | <code>nmap -PS [target]</code> |
| • TCP ACK ping | <code>nmap -PA [target]</code> |
| • UDP ping | <code>nmap -PU [target]</code> |
| • SCTP Init Ping | <code>nmap -PY [target]</code> |
| • ICMP echo ping | <code>nmap -PE [target]</code> |
| • ICMP Timestamp ping | <code>nmap -PP [target]</code> |
| • ICMP address mask ping | <code>nmap -PM [target]</code> |
| • IP protocol ping | <code>nmap -PO [target]</code> |
| • ARP ping | <code>nmap -PR [target]</code> |
| • Traceroute | <code>nmap -traceroute [target]</code> |
| • Force reverse DNS resolution | <code>nmap -R [target]</code> |
| • Disable reverse DNS resolution | <code>nmap -n [target]</code> |
| • Alternative DNS lookup | <code>nmap -system-dns [target]</code> |
| • Manually specify DNS servers | <code>nmap -dns-servers [servers] [target]</code> |
| • Create a host list | <code>nmap -sL [targets]</code> |

NMAP COMMANDS

Firewall Evasion Techniques

- Fragment packets
- Specify a specific MTU
- Use a decoy
- Idle zombie scan
- Manually specify a source port
- Append random data
- Randomize target scan order
- Spoof MAC Address
- Send bad checksums

```
nmap -f [target]
```

```
nmap -mtu [MTU] [target]
```

```
nmap -D RND: [number] [target]
```

```
nmap -sI [zombie] [target]
```

```
nmap -source-port [port] [target]
```

```
nmap -data-length [size] [target]
```

```
nmap -randomize-hosts [target]
```

```
nmap -spoof-mac [MAC|0|vendor] [target]
```

```
nmap -badsum [target]
```

NMAP COMMANDS

Version Detection

- Operating system detection
- Attempt to guess an unknown
- Service version detection
- Troubleshooting version scans
- Perform a RPC scan

```
nmap -O [target]
```

```
nmap -O -ossan-guess [target]
```

```
nmap -sV [target]
```

```
nmap -sV -version-trace [target]
```

```
nmap -sR [target]
```

NMAP COMMANDS

Output Options

- Save output to a text file
- Save output to a xml file
- Grepable output
- Output all supported file types
- Periodically display statistics
- 133t output

```
nmap -oN [scan.txt] [target]
```

```
nmap -oX [scan.xml] [target]
```

```
nmap -oG [scan.txt] [target]
```

```
nmap -oA [path/filename] [target]
```

```
nmap -stats-every [time] [target]
```

```
nmap -oS [scan.txt] [target]
```

NMAP COMMANDS

Basic Scanning Techniques

- Scan a single target
- Scan multiple targets
- Scan a list of targets
- Scan a range of hosts
- Scan an entire subnet
- Scan random hosts
- Excluding targets from a scan
- Excluding targets using a list
- Perform an aggressive scan
- Scan an IPv6 target

```
nmap [target]
```

```
nmap [target1,target2,etc]
```

```
nmap -iL [list.txt]
```

```
nmap [range of IP addresses]
```

```
nmap [IP address/cdir]
```

```
nmap -iR [number]
```

```
nmap [targets] -exclude [targets]
```

```
nmap [targets] -excludefile [list.txt]
```

```
nmap -A [target]
```

```
nmap -6 [target]
```

NMAP MADE EASY

Scanning Multiple Targets

Command:

```
bash  
  
nmap <target1> <target2> ... <targetN>
```

Example:

```
bash  
  
nmap 192.168.1.1 192.168.1.2 192.168.1.3
```

For a range of IPs:

```
bash  
  
nmap 192.168.1.1-50
```

For a subnet:

```
bash  
  
nmap 192.168.1.0/24
```

NMAP MADE EASY

Aggressive Scan

Command:

```
bash  
  
nmap -A <target>
```

Example:

```
bash  
  
nmap -A 192.168.1.1
```

- Enables OS detection, service version detection, script scanning, and traceroute.
- Ideal for gathering comprehensive information about a target.

NMAP MADE EASY

Port Scanning

Scan a specific port:

```
bash  
  
nmap -p <port> <target>
```

Example:

```
bash  
  
nmap -p 80 192.168.1.1
```

Scan multiple ports:

```
bash  
  
nmap -p 22,80,443 192.168.1.1
```

Scan a range of ports:

```
bash  
  
nmap -p 1-1000 192.168.1.1
```

Scan all 65,535 ports:

```
bash  
  
nmap -p- 192.168.1.1
```

NMAP MADE EASY

Service and Version Detection

Command:

```
bash  
  
nmap -sV <target>
```

Example:

```
bash  
  
nmap -sV 192.168.1.1
```

- Detects the versions of services running on open ports.

OS Detection

Command:

```
bash  
  
nmap -O <target>
```

Example:

```
bash  
  
nmap -O 192.168.1.1
```

NMAP MADE EASY

Ping Scan (Host Discovery)

Command:

```
bash  
nmap -sn <target>
```

Example:

```
bash  
nmap -sn 192.168.1.0/24
```

- Pings all devices in the subnet to identify live hosts without scanning ports.

Stealth Scan (SYN Scan)

Command:

```
bash  
nmap -sS <target>
```

Example:

```
bash  
nmap -sS 192.168.1.1
```

- Sends SYN packets to detect open ports without completing the TCP handshake.
- Often used to avoid detection.

NMAP MADE EASY

UDP Scan

Command:

```
bash
nmap -sU <target>
```

Example:

```
bash
nmap -sU 192.168.1.1
```

- Scans for open UDP ports (slower than TCP scans).

Script Scanning

Nmap includes an extensive scripting engine (NSE) for vulnerability detection and more.

Command:

```
bash
nmap --script <script-name> <target>
```

Example:

```
bash
nmap --script http-vuln-cve2017-5638 -p 80 192.168.1.1
```

Run multiple scripts:

```
bash
nmap --script "vuln,*" <target>
```

- Runs all vulnerability-related scripts.

NMAP MADE EASY

Scan and Save Output

Save output to a file:

```
bash  
  
nmap -oN <filename> <target>
```

Example:

```
bash  
  
nmap -oN scan_results.txt 192.168.1.1
```

Save in XML format:

```
bash  
  
nmap -oX <filename.xml> <target>
```

Save in all formats:

```
bash  
  
nmap -oA <basename> <target>
```

Evading Firewalls

Randomize scan order:

```
bash  
  
nmap --randomize-hosts 192.168.1.0/24
```

Use a decoy:

```
bash  
  
nmap -D RND:10 <target>
```

- Launches the scan while masking the source with decoy IPs.

NMAP MADE EASY

Scan Timing Options

Command:

```
bash  
  
nmap -T<0-5> <target>
```

Example:

```
bash  
  
nmap -T4 192.168.1.1
```

- T0 (Paranoid) is the slowest; T5 (Insane) is the fastest.

Scan Specific Hostnames

Command:

```
bash  
  
nmap <hostname>
```

Example:

```
bash  
  
nmap google.com
```

- Resolves the hostname to an IP and scans ↓

NMAP MADE EASY

Scan for Vulnerabilities

Command:

```
bash  
  
nmap --script vuln <target>
```

Example:

```
bash  
  
nmap --script vuln 192.168.1.1
```

- Runs all vulnerability-related scripts.

Scan with Custom IP Address Spoofing

Command:

```
bash  
  
nmap -S <spoofed-IP> <target>
```

Example:

```
bash  
  
nmap -S 10.0.0.1 192.168.1.1
```

- Spoofs the source IP for the scan.

If you are scanning a number of hosts/networks, then you can exclude hosts/IPs from a scan by using:

```
nmap 192.168.1.1-24 -exclude 192.168.1.4
```

```
nmap 192.168.1.1-24 -exclude 192.168.1.3, 192.168.1.7
```

excluding more than one host.

NMAP MADE EASY

Detect Default Credentials

Command:

```
bash  
  
nmap --script http-default-accounts <target>
```

Example:

```
bash  
  
nmap --script http-default-accounts -p 80 192.168.1.1
```

- Checks for default credentials on web services.

Detect Firewall Rules

Command:

```
bash  
  
nmap -sA <target>
```

Example:

```
bash  
  
nmap -sA 192.168.1.1
```

- Determines if a firewall is blocking specific ports.

NMAP MADE EASY

Traceroute

Command:

```
bash  
nmap --traceroute <target>
```

Example:

```
bash  
nmap --traceroute 192.168.1.1
```

- Maps the route packets take to the target.

Scan with Specific Network Interface

Command:

```
bash  
nmap -e <interface> <target>
```

Example:

```
bash  
nmap -e eth0 192.168.1.1
```

- Specifies the network interface for scanning.

NMAP COMPLEX COMMANDS

Find Geo Location of a specific IP address

The following command uses geolocation script "**ip-geolocation-ipinfodb**" to find the geographic location of a specific IP address. To use the above script you need to create a free account at <https://ipinfodb.com/register.php> and get an API key to use in the command as shown below (in script-args)

```
nmap --script=ip-geolocation-ipinfodb --script-args=ip-geolocation-  
ipinfodb.apikey=[APIKEY] 8.8.8.8
```

Nmap scan report for google-public-dns-a.google.com (8.8.8.8)

Host is up (0.0097s latency).

Not shown: 998 filtered ports

PORT	STATE	SERVICE
------	-------	---------

53/tcp	open	domain
--------	------	--------

443/tcp	open	https
---------	------	-------

Host script results:

| ip-geolocation-ipinfodb:

| 8.8.8.8

| coordinates (lat,lon): 37.406,-122.079

| city: Mountain View, California, United States

NMAP COMPLEX COMMANDS

Detect if a Website is protected by WAF

A WAF (Web Application Firewall) can be a software or hardware device in front of web servers to protect from HTTP web application attacks.

The following command uses a script to detect if the target website is protected by a Web Application Firewall (WAF). The **http-waf-detect** script uses two arguments to try the tool's built-in attack vectors for evaluating if the target web domain is protected by a WAF.

```
# nmap -p80,443 --script http-waf-detect --script-args="http-waf-detect.aggro,http-waf-detect.detectBodyChanges" www.networkstraining.com
```

Nmap scan report for www.networkstraining.com (104.18.38.202)

Host is up (0.011s latency).

PORT	STATE	SERVICE
80/tcp	open	http
443/tcp	open	https

| http-waf-detect: IDS/IPS/WAF detected:

|_www.networkstraining.com:443/?p4yl04d=hostname%00

NMAP COMPLEX COMMANDS

Find well known vulnerabilities related to an open port

Let's say you have scanned a target host and found several open services/ports running on the host. With nmap you can query public vulnerability databases to find out if there are any known published vulnerabilities related to the services running.

Step 1:

First you need to download the "nmap-vulners" script from Git and place it under the script directory of nmap:

```
# cd /pentest/vulnerability-analysis/nmap/scripts (or whatever the scripts directory is)
# git clone https://github.com/vulnersCom/nmap-vulners.git
```

Step 2:

Since the script needs to know the exact version of the remote scanned service, you must use the **-sV** key when using the **vulners** script:

```
# nmap -Pn -sV -p80 --script=vulners scanme.nmap.org
```

```
PORT      STATE SERVICE VERSION
80/tcp    open  http   Apache httpd 2.4.7 ((Ubuntu))
|_ http-server-header: Apache/2.4.7 (Ubuntu)
|_ vulners:
|_ cpe:/a:apache:http_server:2.4.7:
|_ CVE-2017-7679 7.5 https://vulners.com/cve/CVE-2017-7679
|_ CVE-2018-1312 6.8 https://vulners.com/cve/CVE-2018-1312
|_ CVE-2014-0226 6.8 https://vulners.com/cve/CVE-2014-0226
|_ CVE-2017-15715 6.8 https://vulners.com/cve/CVE-2017-15715
|_ CVE-2017-9788 6.4 https://vulners.com/cve/CVE-2017-9788
|_ CVE-2013-6438 5.0 https://vulners.com/cve/CVE-2013-6438
|_ CVE-2014-0231 5.0 https://vulners.com/cve/CVE-2014-0231
|_ CVE-2017-9798 5.0 https://vulners.com/cve/CVE-2017-9798
|_ CVE-2016-8743 5.0 https://vulners.com/cve/CVE-2016-8743
|_ CVE-2017-15710 5.0 https://vulners.com/cve/CVE-2017-15710
|_ CVE-2016-0736 5.0 https://vulners.com/cve/CVE-2016-0736
|_ CVE-2014-3523 5.0 https://vulners.com/cve/CVE-2014-3523
|_ CVE-2016-2161 5.0 https://vulners.com/cve/CVE-2016-2161
|_ CVE-2018-17199 5.0 https://vulners.com/cve/CVE-2018-17199
|_ CVE-2014-0098 5.0 https://vulners.com/cve/CVE-2014-0098
|_ CVE-2016-4975 4.3 https://vulners.com/cve/CVE-2016-4975
|_ CVE-2014-0117 4.3 https://vulners.com/cve/CVE-2014-0117
|_ CVE-2014-8109 4.3 https://vulners.com/cve/CVE-2014-8109
|_ CVE-2015-3185 4.3 https://vulners.com/cve/CVE-2015-3185
|_ CVE-2014-0118 4.3 https://vulners.com/cve/CVE-2014-0118
|_ CVE-2018-1283 3.5 https://vulners.com/cve/CVE-2018-1283
|_ CVE-2016-8612 3.3 https://vulners.com/cve/CVE-2016-8612
```

As you can see from above, we have scanned port 80 (with **-sV** switch) and used the **vulners** script to get all known public vulnerabilities of the specific service (Apache httpd 2.4.7).

NMAP COMPLEX COMMANDS

Scan the most popular ports

Using “--top-ports” parameter along with a specific number lets you scan the top X most common ports for that host, as we can see:

```
nmap --top-ports 20 192.168.1.106
```

Replace “20” with the desired number. Output example:

```
[root@securitytrails:~]nmap --top-ports 20 localhost
Starting Nmap 6.40 at 2018-10-01 10:02 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000016s latency).
Other addresses for localhost (not scanned): 127.0.0.1
PORT STATE SERVICE
21/tcp closed ftp
22/tcp closed ssh
23/tcp closed telnet
25/tcp closed smtp
53/tcp closed domain
80/tcp filtered http
110/tcp closed pop3
111/tcp closed rpcbind
135/tcp closed msrpc
139/tcp closed netbios-ssn
143/tcp closed imap
443/tcp filtered https
445/tcp closed microsoft-ds
993/tcp closed imaps
995/tcp closed pop3s
1723/tcp closed pptp
3306/tcp closed mysql
3389/tcp closed ms-wbt-server
5900/tcp closed vnc
8080/tcp closed http-proxy
```

NMAP COMPLEX COMMANDS

Detect service/daemon versions

This can be done by using -sV parameters

```
nmap -sV localhost
```

As you can see here:

```
[root@securitytrails:~]nmap -sV localhost
Starting Nmap 7.60 at 2018-10-01 09:28 -03
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000020s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 997 closed ports
PORT STATE SERVICE VERSION
111/tcp open rpcbind 2-4 (RPC #100000)
631/tcp open ipp CUPS 2.2
902/tcp open ssl/vmware-auth VMware Authentication Daemon 1.10 (Uses VNC,
SOAP)

Service detection performed. Please report any incorrect results at https:// nmap
.org/submit/ . Nmap done: 1 IP address (1 host up) scanned in 7.96 seconds
```

NMAP COMPLEX COMMANDS

Testing brute force attacks

NSE is really fascinating -- it contains scripts for everything you can imagine. See the next three examples of BFA against WordPress, MSSQL, and FTP server:

```
nmap -sV --script http-wordpress-brute --script-args  
'userdb=users.txt,passdb=passwd.txt,http-wordpress-  
brute.hostname=domain.xyz, http-wordpress-  
brute.threads=3,brute.firstonly=true' 192.168.1.105
```

WordPress brute force attack:

```
nmap -p 1433 --script ms-sql-brute --script-args  
userdb=customuser.txt,passdb=custompass.txt 192.168.1.105
```

Brute force attack against MS-SQL:

```
nmap -p 1433 --script ms-sql-brute --script-args  
userdb=customuser.txt,passdb=custompass.txt 192.168.1.105
```

FTP brute force attack:

```
nmap --script ftp-brute -p 21 192.168.1.105
```

NMAP COMPLEX COMMANDS

Detecting malware infections on remote hosts

Nmap is able to detect malware and backdoors by running extensive tests on a few popular OS services like Identd, Proftpd, Vsftpd, IRC, SMB, and SMTP. It also has a module to check for popular malware signs inside remote servers and integrates Google's Safe Browsing and VirusTotal databases as well.

A common malware scan can be performed by using:

```
nmap -sV --script=http-malware-host 192.168.1.105
```

Or using Google's Malware check:

```
nmap -p80 --script http-google-malware infectedsite.xyz
```

Output example:

Nmap is one of the most complete and accurate port scanners used by infosec professionals today. With it, you can perform simple port scan tasks or use its powerful scripting engine to launch DOS attacks, detect malware or brute force testing on remote and local servers.

NMAP COMPLEX COMMANDS

Flood testing with Nmap

Nmap allows to launch flood attacks against network to test how effective mitigation methods are

```
nmap 192.168.1.105 -max-parallelism 800 -Pn --  
script http-slowloris --script-args http-  
slowloris.runforever=true
```

NMAP OTHER OPTIONS

COMPARISON TABLE

Tool	Speed	Features	Complexity	Use Case	Free/Paid
Nmap	Moderate	Comprehensive	Moderate	General-purpose scanning	Free
Zenmap	High	Comprehensive	Low	Comprehensive	Free
Angry IP Scanner	Moderate	Basic scanning	Low	Quick scans of small networks	Free
OpenVAS	Moderate	Vulnerability scanning	High	Detailed vulnerability analysis	Free (Community)
Nessus	Moderate	Vulnerability scanning	High	Enterprise security audits	Paid
Nikto	Low	Web server scanning	Moderate	Web server-specific audits	Free
RustScan	High	Port scanning	Low	Speedy scans with deeper analysis	Free